SENATI

Escuela de Tecnologías de la Información

GUÍA DE LABORATORIO

Guía 10

Aprendizaje no supervisado: Algoritmo K-Means

Crea programas con algoritmos de aprendizaje no supervisado

Escuela	Tecnologías de la Información
Carrera	Ingeniería de Software con Inteligencia Artificial
Curso	Seminario de Complementación Práctica I (PIAD-427)
Semana	10
Tarea de aprendizaje	Crea programas con algoritmos de aprendizaje no supervisado
Herramientas	Python 3 · Visual Studio Code · scikit-learn
Instructor	Rembrandt Ubalde Enríquez
Estudiante	
Fecha	

Contenido

Objetivos de la guía 3
Materiales 3
Parte 1. Preparación del entorno en Visual Studio Code 4
1.1 Crear la carpeta del proyecto 4
1.2 Crear y activar el entorno virtual 4
1.3 Instalar las librerías 4
1.4 Seleccionar el intérprete y ejecutar scripts 4
Parte 2. Conceptos clave 6
2.1 ¿Qué es el aprendizaje no supervisado? 6
2.2 Tipos de algoritmos no supervisados 6
2.3 Diferencias entre clasificación y agrupamiento 6
2.4 ¿Cómo funciona K-Means? 6
2.5 Implementación del algoritmo K-Means 7
2.6 Intervalos de confianza (conocimiento complementario) 7
Parte 3. Prácticas en Visual Studio Code 8
Práctica 01. Los tres tipos de aprendizaje no supervisado 8
Práctica 02. Clasificación vs. agrupamiento 8
Práctica 03. K-Means paso a paso con NumPy 9
Práctica 04. Segmentación de clientes con scikit-learn 10
Práctica 05. ¿Cuántos grupos elegir? Codo y silueta 13
Práctica 06. Intervalos de confianza por grupo (complementario) 14
Parte 4. Errores frecuentes 15
Parte 5. Ejercicios propuestos 15
Preguntas de reflexión 16
Lista de cotejo 16

(Si el índice aparece vacío, haz clic derecho sobre él y elige «Actualizar campos».)

Objetivos de la guía

Al terminar esta guía serás capaz de:

- **Describir** los tipos de algoritmos de aprendizaje no supervisado: agrupamiento, reducción de la dimensionalidad y detección de anomalías.
- **Definir** las diferencias entre algoritmos de clasificación y de agrupamiento.
- **Implementar** el algoritmo K-Means con Python en Visual Studio Code y elegir el número de grupos.
- **Interpretar** intervalos de confianza y su uso en la inferencia estadística.

Operación	Dónde se trabaja en esta guía
Describe los tipos de algoritmos de aprendizaje no supervisado	Parte 2 (secciones 2.1 y 2.2) y Práctica 01
Define las diferencias entre algoritmos de clasificación y agrupamiento	Parte 2 (sección 2.3) y Práctica 02
Implementa algoritmo K-Means con Python	Parte 2 (sección 2.4) y Prácticas 03 a 05
Intervalos de confianza (complementario)	Parte 2 (sección 2.6) y Práctica 06

Materiales

- Computadora con Windows 10/11, Python 3.10 o superior y Visual Studio Code con las extensiones **Python** y **Pylance**.
- Conexión a internet para instalar las librerías (solo la primera vez).

Parte 1. Preparación del entorno en Visual Studio Code

Si ya trabajaste las guías anteriores, tienes Python y VS Code listos. Igual sigue estos pasos para crear la **carpeta de esta guía** y su **entorno virtual**.

1.1 Crear la carpeta del proyecto

1. Crea en Documentos una carpeta llamada **IA_Guia10_KMeans**.
2. Abre Visual Studio Code y entra a **Archivo > Abrir carpeta…** (Ctrl + K, Ctrl + O). Selecciona la carpeta.
3. Si VS Code pregunta **«¿Confía en los autores de los archivos?»**, elige **Sí, confío en los autores**.
4. Abre la terminal integrada con **Terminal > Nuevo terminal** (Ctrl + ñ).

1.2 Crear y activar el entorno virtual

Terminal de VS Code

```
python -m venv venv
.\venv\Scripts\activate
```

Cuando el entorno está activo, la terminal muestra **(venv)** al inicio de la línea.

> ⚠ **Atención:** Si PowerShell muestra **«la ejecución de scripts está deshabilitada en este sistema»**, ejecuta una sola vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, responde **S** y vuelve a activar el entorno.

1.3 Instalar las librerías

Terminal de VS Code

```
pip install numpy pandas matplotlib scikit-learn scipy
```

Verifica la instalación:

Terminal de VS Code

```
python -c "import sklearn, scipy; print(sklearn.__version__, scipy.__version__)"
```

▶ Resultado esperado en la terminal

```
1.x.x 1.x.x   (o versiones más recientes)
```

1.4 Seleccionar el intérprete y ejecutar scripts

1. Pulsa **Ctrl + Shift + P**, escribe **Python: Select Interpreter** y elige el que dice **('venv': venv)**.
2. Crea cada archivo con el ícono **Nuevo archivo** del Explorador, con el nombre exacto que indica la guía (por ejemplo `01_tipos_supervisado.py`).
3. Copia el código y guarda con **Ctrl + S**.

4. Ejecuta con el botón ▶ **Run Python File** (arriba a la derecha) o escribiendo `python nombre_del_archivo.py` en la terminal.
5. Compara lo que aparece en la terminal con el recuadro verde **Resultado esperado**.

> 💡 **Consejo:** Cuando un script usa `plt.show()`, el gráfico se abre en una **ventana aparte** y el programa queda en pausa hasta que **cierres esa ventana**. El gráfico también queda guardado como imagen PNG en tu carpeta gracias a `plt.savefig(...)`.

> 💡 **Consejo:** Para depurar paso a paso: haz clic a la izquierda del número de línea para poner un **punto de interrupción** (punto rojo) y pulsa **F5 > Python File**. En el panel **Variables** verás los valores de `X`, `y`, `modelo`, etc. mientras el programa avanza.

Parte 2. Conceptos clave

2.1 ¿Qué es el aprendizaje no supervisado?

En el **aprendizaje no supervisado** los datos **no tienen etiqueta**: no hay una respuesta correcta que el modelo deba imitar. El algoritmo explora los datos por sí solo y descubre **estructuras ocultas**, como grupos de elementos parecidos o datos fuera de lo normal. Es como ordenar una caja de piezas de LEGO sin instrucciones: las juntas por color o forma porque se parecen.

2.2 Tipos de algoritmos no supervisados

Tipo	¿Qué hace?	Algoritmos	Ejemplo de uso
Agrupamiento (Clustering)	Forma grupos de datos similares entre sí y diferentes de los otros grupos.	K-Means, DBSCAN, Jerárquico	Segmentar clientes de un supermercado.
Reducción de la dimensionalidad	Reduce el número de columnas conservando la mayor parte de la información.	PCA, t-SNE	Visualizar en 2D datos de 50 columnas.
Detección de anomalías	Encuentra datos raros que no siguen el patrón general.	Isolation Forest, One-Class SVM	Detectar fraudes con tarjetas de crédito.

2.3 Diferencias entre clasificación y agrupamiento

Aspecto	Algoritmos de clasificación	Algoritmos de agrupamiento
Tipo de aprendizaje	Supervisado	No supervisado
Datos de entrenamiento	**Con etiquetas** (se conoce la clase).	**Sin etiquetas**.
Grupos	Definidos **antes** de entrenar (spam / no spam).	**Descubiertos** por el algoritmo; hay que interpretarlos.
Objetivo	Asignar un dato nuevo a una clase conocida.	Encontrar la estructura natural de los datos.
Evaluación	Exactitud, precisión, recall.	Inercia, coeficiente de silueta.
Algoritmos	KNN, Árbol de decisión, SVM, Regresión logística.	K-Means, DBSCAN, Jerárquico.

2.4 ¿Cómo funciona K-Means?

K-Means divide los datos en **k grupos**. Cada grupo se representa con un **centroide** (su punto central). El algoritmo repite cuatro pasos:

1. **Inicialización:** elige k centroides iniciales (scikit-learn usa por defecto **k-means++**, que los reparte de forma inteligente).
2. **Asignación:** cada punto se asigna al centroide más cercano según la **distancia euclidiana**.
3. **Actualización:** cada centroide se mueve al **promedio** de los puntos de su grupo.

4. **Convergencia:** se repiten los pasos 2 y 3 hasta que los centroides ya no se mueven.

Concepto	Significado
k	Número de grupos. **Lo decide la persona**, no el algoritmo.
Inercia	Suma de las distancias al cuadrado de cada punto a su centroide. Mientras más baja, más compactos los grupos.
Método del codo	Se grafica la inercia para varios k; el mejor k está donde la curva «dobla» y deja de bajar mucho.
Coeficiente de silueta	Mide qué tan bien separado está cada grupo. Va de -1 a 1; más alto es mejor.
Escalado	K-Means usa distancias: si una columna tiene valores grandes (ingresos) y otra pequeños (edad), hay que **escalarlas** con `StandardScaler`.

2.5 Implementación del algoritmo K-Means

Etapa	Código clave
1. Preparar y escalar los datos	`StandardScaler().fit_transform(df)`
2. Elegir k (codo y silueta)	`km.inertia_`, `silhouette_score`
3. Entrenar el modelo	`KMeans(n_clusters=k).fit(X)`
4. Interpretar grupos y asignar datos nuevos	`km.cluster_centers_`, `km.predict()`

2.6 Intervalos de confianza (conocimiento complementario)

En estadística casi nunca conocemos el valor real de toda la población; trabajamos con una **muestra**. Un **intervalo de confianza (IC)** es un rango de valores que, con un nivel de confianza dado (por ejemplo 95 %), contiene al valor real de la población.

Fórmula del IC para la media (muestras pequeñas o σ desconocida)

```
IC = x̄  ±  t · ( s / √n )

x̄ = media de la muestra      s = desviación estándar
n = tamaño de la muestra     t = valor de la tabla t de Student
```

- **Interpretación correcta:** «Si repitiéramos el muestreo muchas veces, el 95 % de los intervalos construidos contendría la media real».
- Un intervalo **más angosto** indica una estimación más precisa; se logra con **más datos** o con datos menos dispersos.
- **Uso en la inferencia:** si los intervalos de dos grupos **no se superponen**, hay evidencia de que sus medias son diferentes.

Parte 3. Prácticas en Visual Studio Code

Práctica 01. Los tres tipos de aprendizaje no supervisado

Usarás el dataset **Iris** pero **sin sus etiquetas**, para aplicar un algoritmo de cada tipo.

Archivo: 01_tipos_no_supervisado.py

```
# 01_tipos_no_supervisado.py - Los 3 tipos de aprendizaje no supervisado
import numpy as np
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.ensemble import IsolationForest

X = load_iris().data          # SOLO las medidas: NO usamos las etiquetas (target)
print("Datos sin etiquetas:", X.shape)

# 1. AGRUPAMIENTO (Clustering): forma grupos de flores parecidas
grupos = KMeans(n_clusters=3, n_init=10, random_state=42).fit_predict(X)
print("\n1) Clustering -> flores por grupo:", np.bincount(grupos))

# 2. REDUCCIÓN DE LA DIMENSIONALIDAD: de 4 columnas a 2
pca = PCA(n_components=2)
X2 = pca.fit_transform(X)
print("2) PCA -> forma nueva:", X2.shape,
      "| información conservada:", f"{pca.explained_variance_ratio_.sum():.1%}")

# 3. DETECCIÓN DE ANOMALÍAS: encuentra flores "raras"
detector = IsolationForest(contamination=0.03, random_state=42)
marcas = detector.fit_predict(X)                # -1 = anomalía, 1 = normal
print("3) Anomalías encontradas:", (marcas == -1).sum(),
      "en las filas", np.where(marcas == -1)[0])
```

▶ Resultado esperado en la terminal

```
Datos sin etiquetas: (150, 4)

1) Clustering -> flores por grupo: [62 50 38]
2) PCA -> forma nueva: (150, 2) | información conservada: 97.8%
3) Anomalías encontradas: 5 en las filas [ 15  41 117 118 131]
```

PCA redujo las 4 medidas a 2 columnas y aun así conservó el **97.8 %** de la información. Isolation Forest marcó 5 flores con medidas poco comunes.

Práctica 02. Clasificación vs. agrupamiento

Aplicarás un clasificador (supervisado) y K-Means (no supervisado) al mismo dataset para ver la diferencia.

Archivo: 02_clasificacion_vs_agrupamiento.py

```
# 02_clasificacion_vs_agrupamiento.py - Mismo dataset, dos enfoques distintos
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.cluster import KMeans

iris = load_iris()
X, y = iris.data, iris.target

# CLASIFICACIÓN (supervisado): aprende de ejemplos CON etiqueta
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=0)
arbol = DecisionTreeClassifier(random_state=0).fit(X_tr, y_tr)
print("CLASIFICACIÓN (Árbol de decisión)")
print("  ¿Usa etiquetas al entrenar?: Sí")
print(f"  Exactitud en prueba: {arbol.score(X_te, y_te):.2%}")

# AGRUPAMIENTO (no supervisado): descubre grupos SIN etiqueta
km = KMeans(n_clusters=3, n_init=10, random_state=42).fit(X)
print("\nAGRUPAMIENTO (K-Means)")
print("  ¿Usa etiquetas al entrenar?: No")
print("  Grupos descubiertos vs. especie real:")
tabla = pd.crosstab(pd.Series(iris.target_names[y], name="especie_real"),
                    pd.Series(km.labels_, name="grupo"))
print(tabla)
```

▶ Resultado esperado en la terminal

```
CLASIFICACIÓN (Árbol de decisión)
  ¿Usa etiquetas al entrenar?: Sí
  Exactitud en prueba: 97.78%

AGRUPAMIENTO (K-Means)
  ¿Usa etiquetas al entrenar?: No
  Grupos descubiertos vs. especie real:
grupo          0   1   2
especie_real
setosa         0  50   0
versicolor    48   0   2
virginica     14   0  36
```

Análisis: sin conocer las especies, K-Means separó **perfectamente** a setosa (50 flores en el grupo 1). Versicolor y virginica se mezclan un poco porque sus medidas se parecen. Nota que K-Means **no sabe los nombres** de los grupos: solo los numera; interpretarlos es tarea nuestra.

Práctica 03. K-Means paso a paso con NumPy

Programa tú mismo los pasos del algoritmo (sección 2.4) con 8 puntos y k = 2.

Archivo: 03_kmeans_paso_a_paso.py

```
# 03_kmeans_paso_a_paso.py - Cómo "piensa" K-Means, programado con NumPy
import numpy as np

puntos = np.array([[1, 2], [1.5, 1.8], [1, 0.6], [5, 8],
                   [8, 8], [9, 11], [8, 2], [9, 3]])
k = 2
centroides = puntos[[0, 3]].astype(float)       # 1) INICIALIZACIÓN: k centroides
print("Centroides iniciales:\n", centroides)

for it in range(1, 11):
    # 2) ASIGNACIÓN: cada punto va al centroide más cercano (distancia euclidiana)
    distancias = np.linalg.norm(puntos[:, None] - centroides[None, :], axis=2)
    grupo = distancias.argmin(axis=1)
    # 3) ACTUALIZACIÓN: cada centroide pasa al promedio de sus puntos
    nuevos = np.array([puntos[grupo == j].mean(axis=0) for j in range(k)])
    print(f"Iteración {it}: grupos = {grupo} | centroides =
{nuevos.round(2).tolist()}")
    # 4) CONVERGENCIA: si los centroides ya no se mueven, terminamos
    if np.allclose(nuevos, centroides):
        print("Los centroides no cambiaron: el algoritmo CONVERGIÓ.")
        break
    centroides = nuevos
```

▶ Resultado esperado en la terminal

```
Centroides iniciales:
 [[1. 2.]
 [5. 8.]]
Iteración 1: grupos = [0 0 0 1 1 1 1 1] | centroides = [[1.17, 1.47], [7.8, 6.4]]
Iteración 2: grupos = [0 0 0 1 1 1 1 1] | centroides = [[1.17, 1.47], [7.8, 6.4]]
Los centroides no cambiaron: el algoritmo CONVERGIÓ.
```

> 💡 **Consejo:** Pon un punto de interrupción en la línea `grupo = distancias.argmin(axis=1)` y ejecuta con **F5**. En el panel Variables verás la matriz de distancias de cada punto a cada centroide.

Práctica 04. Segmentación de clientes con scikit-learn

Un centro comercial quiere agrupar a 200 clientes según su **ingreso anual** y su **puntaje de gasto** (1 a 100) para diseñar campañas. El script crea el archivo **clientes.csv**, que usarán las Prácticas 05 y 06.

Archivo: 04_kmeans_clientes.py

```
# 04_kmeans_clientes.py - Segmentación de clientes con K-Means (scikit-learn)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# 1. PREPARACIÓN DE DATOS (dataset simulado de un centro comercial)
X, _ = make_blobs(n_samples=200, centers=[[25, 20], [30, 80], [70, 50], [95, 20], [95,
85]],
                  cluster_std=[6, 6, 7, 6, 6], random_state=3)
df = pd.DataFrame({"ingreso_mil": X[:, 0].round(1), "puntaje_gasto": X[:, 1].clip(1,
100).round(0)})
df.to_csv("clientes.csv", index=False)
print(df.describe().round(1))

escalador = StandardScaler()                # pone ambas columnas en la misma escala
X_esc = escalador.fit_transform(df)

# 2. ENTRENAMIENTO
km = KMeans(n_clusters=5, init="k-means++", n_init=10, random_state=42)
df["grupo"] = km.fit_predict(X_esc)

# 3. RESULTADOS
centros = pd.DataFrame(escalador.inverse_transform(km.cluster_centers_),
                       columns=["ingreso_mil", "puntaje_gasto"]).round(1)
centros["clientes"] = df["grupo"].value_counts().sort_index().values
print("\nCentroides de cada grupo (en escala original):")
print(centros)
print(f"\nInercia (suma de distancias²): {km.inertia_:.2f}")

nuevo = pd.DataFrame({"ingreso_mil": [90], "puntaje_gasto": [88]})
print("Cliente nuevo (90 mil, gasto 88) -> grupo",
km.predict(escalador.transform(nuevo))[0])

plt.scatter(df["ingreso_mil"], df["puntaje_gasto"], c=df["grupo"], cmap="tab10", s=30)
plt.scatter(centros["ingreso_mil"], centros["puntaje_gasto"], c="black", marker="X",
s=200, label="Centroides")
plt.title("Segmentación de clientes con K-Means (k = 5)")
plt.xlabel("Ingreso anual (miles de S/)"); plt.ylabel("Puntaje de gasto (1-100)")
plt.legend(); plt.grid(alpha=0.3)
plt.savefig("g10_clientes.png", dpi=110, bbox_inches="tight")
plt.show()
```

▶ Resultado esperado en la terminal

```
       ingreso_mil  puntaje_gasto
count        200.0          200.0
mean          63.0           51.0
std           31.9           28.8
min           11.5            5.0
25%           29.0           22.8
50%           69.1           50.0
75%           94.2           81.0
max          111.3           97.0

Centroides de cada grupo (en escala original):
   ingreso_mil  puntaje_gasto  clientes
0         96.5           20.0        40
1         28.6           80.8        40
2         70.4           50.7        40
3         24.6           19.6        40
4         94.9           83.6        40

Inercia (suma de distancias²): 18.08
Cliente nuevo (90 mil, gasto 88) -> grupo 4
```

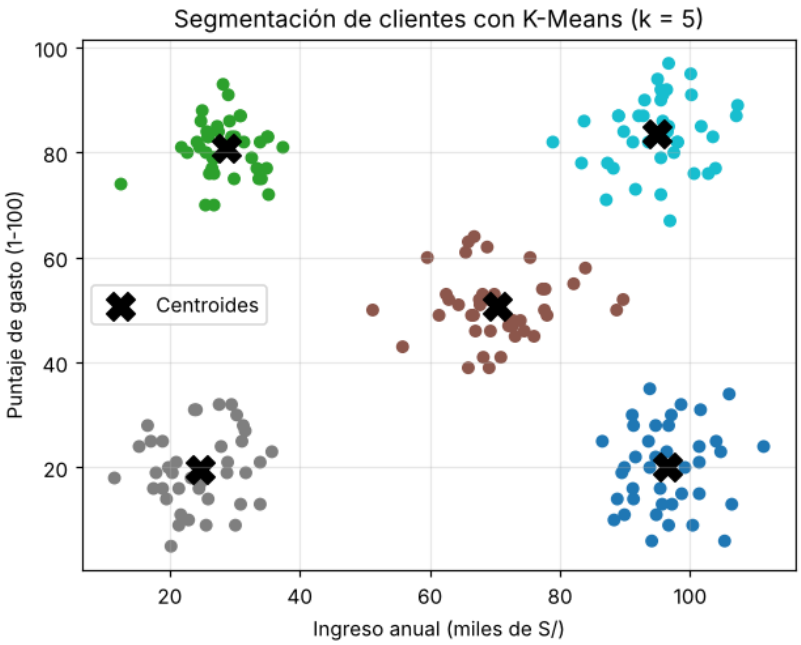


Figura 1. Cinco segmentos de clientes y sus centroides (X negras).

Grupo	Perfil	Estrategia sugerida
0	Ingreso alto, gasto bajo	Clientes cautelosos: ofertas de calidad y garantía.
1	Ingreso bajo, gasto alto	Promociones y pagos en cuotas.
2	Ingreso y gasto medios	Programa de puntos para fidelizar.
3	Ingreso bajo, gasto bajo	Descuentos en productos básicos.
4	Ingreso alto, gasto alto	Clientes VIP: atención exclusiva y lanzamientos.

⚠ **Atención:** El **número** de cada grupo puede cambiar según la versión de scikit-learn instalada (por ejemplo, el grupo VIP puede salir como 2 en vez de 4). Lo importante es el **perfil** de cada centroide, no su número.

Práctica 05. ¿Cuántos grupos elegir? Codo y silueta

Archivo: 05_elegir_k.py

```
# 05_elegir_k.py - ¿Cuántos grupos? Método del codo y coeficiente de silueta
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

df = pd.read_csv("clientes.csv")
X = StandardScaler().fit_transform(df)

inercias = []
print(" k   inercia   silueta")
for k in range(2, 10):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    inercias.append(km.inertia_)
    print(f"{k:>2} {km.inertia_:>9.2f} {silhouette_score(X, km.labels_):>9.3f}")

plt.plot(range(2, 10), inercias, marker="o")
plt.title("Método del codo")
plt.xlabel("Número de grupos (k)"); plt.ylabel("Inercia")
plt.grid(alpha=0.3)
plt.savefig("g10_codo.png", dpi=110, bbox_inches="tight")
plt.show()
```

▶ Resultado esperado en la terminal

```
 k   inercia   silueta
 2    223.65     0.454
 3    133.52     0.553
 4     54.18     0.650
 5     18.08     0.734
 6     15.95     0.673
 7     14.26     0.592
 8     12.73     0.511
 9     11.28     0.459
```

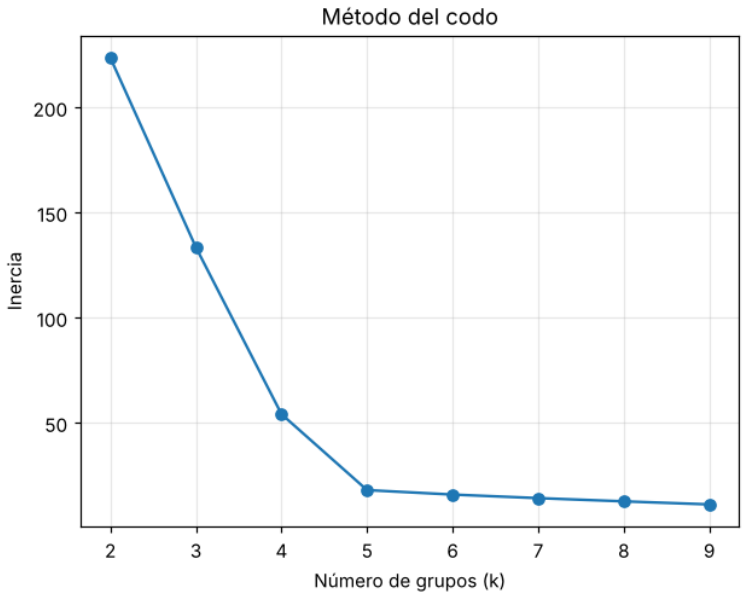


Figura 2. Método del codo: la curva deja de bajar fuertemente en k = 5.

Ambos criterios coinciden: en **k = 5** la inercia deja de bajar bruscamente (el «codo») y la silueta alcanza su valor **máximo (0.734)**. Por eso en la Práctica 04 usamos 5 grupos.

Práctica 06. Intervalos de confianza por grupo (complementario)

Calcularás el IC del 95 % para el puntaje de gasto promedio de cada segmento con **SciPy**.

Archivo: 06_intervalos_confianza.py

```
# 06_intervalos_confianza.py - Intervalo de confianza del 95 % para la media de cada
grupo
import numpy as np
import pandas as pd
from scipy import stats
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

df = pd.read_csv("clientes.csv")
X = StandardScaler().fit_transform(df)
df["grupo"] = KMeans(n_clusters=5, n_init=10, random_state=42).fit_predict(X)

print("IC 95 % para el puntaje de gasto promedio de cada grupo")
print("grupo  n   media   límite_inf  límite_sup")
for g, datos in df.groupby("grupo")["puntaje_gasto"]:
    n = len(datos)
    media = datos.mean()
    error_est = stats.sem(datos)                        # s / raíz(n)
    li, ls = stats.t.interval(0.95, df=n - 1, loc=media, scale=error_est)
    print(f"{g:>4} {n:>4} {media:>7.2f} {li:>11.2f} {ls:>11.2f}")
```

▶ **Resultado esperado en la terminal**

```
IC 95 % para el puntaje de gasto promedio de cada grupo
grupo  n   media   límite_inf  límite_sup
   0   40   20.00       17.49       22.51
   1   40   80.78       79.10       82.45
   2   40   50.67       48.65       52.70
   3   40   19.65       17.39       21.91
   4   40   83.65       81.45       85.85
```

Interpretación: con 95 % de confianza, el gasto promedio real del grupo 4 está entre **81.45 y 85.85**. Los intervalos de los grupos 0 y 3 (alrededor de 20) **se superponen**: no hay evidencia de que su gasto promedio sea diferente; lo que los separa es el ingreso. En cambio, el intervalo del grupo 2 (48.65 a 52.70) no toca a ningún otro: su gasto es claramente distinto.

Parte 4. Errores frecuentes

Mensaje o síntoma	Causa	Solución
`ModuleNotFoundError: No module named 'scipy'`	Falta instalar la librería en el venv.	Activa el venv y ejecuta `pip install scipy`.
`FileNotFoundError: clientes.csv`	No se ejecutó la Práctica 04.	Ejecuta `04_kmeans_clientes.py` primero.
Advertencia sobre `n_init`	Versiones antiguas de scikit-learn piden indicar `n_init`.	Deja `n_init=10` como en el código de la guía.
Los grupos salen con otro número	El orden de los grupos es arbitrario.	Interpreta los grupos por sus centroides.
Grupos «raros» o alargados	No se escalaron los datos.	Aplica `StandardScaler` antes de K-Means.
«No se encontró Python; …Microsoft Store»	Python no está en el PATH.	Instala Python marcando **Add python.exe to PATH** y reinicia VS Code.

Parte 5. Ejercicios propuestos

1. En la Práctica 04 ejecuta K-Means **sin** `StandardScaler`. Compara los centroides con los originales y explica la diferencia.
2. Cambia `n_clusters` a 3 y a 7 en la Práctica 04. ¿Los grupos siguen teniendo sentido para el negocio?
3. Agrega a la Práctica 03 un tercer centroide (k = 3) y analiza en cuántas iteraciones converge.
4. Aplica K-Means a un dataset propio de al menos 50 filas (por ejemplo, notas de dos cursos de tus compañeros, con datos ficticios) y describe cada grupo.
5. Calcula en la Práctica 06 el intervalo de confianza del 90 % y del 99 %. ¿Cómo cambia el ancho del intervalo?

Preguntas de reflexión

1. ¿Por qué K-Means se considera un algoritmo **no supervisado**?
2. ¿Qué ventaja tiene la clasificación sobre el agrupamiento y en qué casos solo es posible usar agrupamiento?
3. ¿Por qué es necesario escalar los datos antes de aplicar K-Means?
4. Explica con tus palabras qué significa un intervalo de confianza del 95 %.

Lista de cotejo

☐ Creé la carpeta, el entorno virtual e instalé las librerías.

☐ Describo los tres tipos de aprendizaje no supervisado con un ejemplo de cada uno (Práctica 01).

☐ Explico las diferencias entre clasificación y agrupamiento (Práctica 02).

☐ Programé los pasos de K-Means con NumPy (Práctica 03).

☐ Segmenté clientes con scikit-learn e interpreté los centroides (Práctica 04).

☐ Elegí k con el método del codo y el coeficiente de silueta (Práctica 05).

☐ Calculé e interpreté intervalos de confianza (Práctica 06).

☐ Resolví al menos tres ejercicios propuestos.